Coding Assignments

```
$ python grade.py
A1 Matrix functions          10.0 / 10
A2 Vector derivatives        15.0 / 15
A3 Matrix derivatives        20.0 / 20
A4 Backpropagation           20.0 / 20
A5 Autodiff                  20.0 / 20
A6 Deep-learning layers      15.0 / 15
TOTAL                       100.0 / 100
```

Matrix calculus for deep learning:
six graded assignments in NumPy

Part 3 of a three-part course companion to Bishop & Bishop, *Deep Learning: Foundations and Concepts*

1. **Textbook** — derivations by hand, worked examples, exercises with full solutions.
2. **NumPy Lab Manual** — every derivative coded and checked numerically; backprop and autodiff from scratch.
3. **Coding Assignments** (this handout) — six graded assignments with an automatic `pytest` test suite.

Edition of October 2026

Contents

1 Getting Started **1**

1.1 What you will build 1

1.2 Setting up 1

1.3 The rules 2

1.4 Running the tests 2

1.5 Grading 3

1.6 How to work 3

2 A1 — Gradient Checking, Matrix Functions, Identities **4**

3 A2 — Vector Derivatives and Optimisation **6**

4 A3 — Matrix Derivatives and Gaussians **8**

5 A4 — Backpropagation **10**

6 A5 — Automatic Differentiation **12**

7 A6 — Attention, Normalisation, Flows **14**

A Troubleshooting **16**

B The Test Files **17**

CHAPTER 1

Getting Started

1.1 What you will build

Across six assignments you write, in plain NumPy, the matrix calculus that Bishop & Bishop use on almost every page: a gradient checker; log-determinants, matrix square roots and exponentials; the Woodbury and Schur-complement identities; gradients, Jacobians and Hessians of losses; derivatives with respect to matrices and the Gaussian maximum-likelihood equations; a multilayer perceptron with a hand-written backward pass; a forward-mode and a reverse-mode automatic-differentiation engine; and the backward passes of self-attention, layer and batch normalisation and normalising flows.

The test suite checks each function against three things: the textbook's hand calculations, NumPy (where NumPy has an answer), and *finite differences* computed by the tests' own independent code. A wrong gradient therefore cannot pass, however plausible it looks.

1.2 Setting up

You need Python 3.10 or newer. Unzip `matcalc_assignments_starter.zip`, then

```
cd matcalc-assignments
python -m pip install -r requirements.txt
python grade.py
```

A fresh copy prints a score of **0 / 100**, with every test listed as "not done". The folder contains:

```
mc_assignments/       YOUR CODE: six modules with functions marked TODO
    a1_matrix_functions.py    a2_vector_derivatives.py   a3_matrix_derivatives.py
    a4_backprop.py            a5_autodiff.py             a6_layers.py
tests/                the test suite (112 tests) -- do not edit
grade.py              runs the tests and prints your score per assignment
README.md, requirements.txt, pytest.ini, ASSIGNMENTS.pdf (this handout)
```

1.3 The rules

1. **NumPy only.** `numpy.linalg` (`solve`, `inv`, `det`, `slogdet`, `cholesky`, `eigh`, `eigvalsh`, `norm`) is allowed: the subject here is derivatives, not linear-algebra algorithms.
2. **Nothing that differentiates for you.** No `jax`, `torch`, `tensorflow`, `autograd`, and also no `scipy`, `sklearn`, or the Lab Manual's `mcalc` module. A test reads your source code and fails on any such import.
3. **Gradients have the shape of the variable.** The gradient with respect to a 3×4 matrix is a 3×4 matrix; a Jacobian of $f : \mathbb{R}^n \to \mathbb{R}^m$ is $m \times n$ (Textbook Chapter 1).
4. **Every derivative is your own formula.** Functions that return a gradient must not compute it numerically; the finite-difference tools of A1 are for *checking*.
5. The **docstring of each function is its specification**: argument shapes, return values, conventions, stability requirements. Read it before you start. You may add private helpers (names starting with _).
6. Do not edit the tests.

1.4 Running the tests

```
python grade.py                    # score report for all six assignments
python grade.py a3                 # only Assignment 3
python grade.py a4 -v              # Assignment 4 with the failure messages
pytest tests/test_a2_vector_derivatives.py -x     # stop at the first failure
pytest -k "attention and not causal" -v           # tests selected by name
```

A score report looks like this (for a student who has finished A1 and A2):

```
================== Matrix-calculus assignments: score report ==================
Assignment                              passed  failed  not done        points
A1  Matrix functions & checking             17       0         0     10.0 / 10
A2  Vector derivatives                      15       0         0     15.0 / 15
A3  Matrix derivatives & Gaussians           0       0        20      0.0 / 20
A4  Backpropagation                          0       0        17      0.0 / 20
A5  Automatic differentiation                0       0        24      0.0 / 20
A6  Deep-learning layers                     0       0        19      0.0 / 15
-------------------------------------------------------------------------------
TOTAL                                                                25.0 / 100
```

"Not done" counts tests that hit `NotImplementedError`. Within an assignment every test carries equal weight. The `test_rules` test of each module (no forbidden imports) waits until every function and method of that module is implemented. Each test has a time limit of 60 seconds; the whole suite runs in a few seconds.

1.5 Grading

	Assignment	Tests	Points	Textbook / Lab
A1	Gradient checking, matrix functions, identities	17	10	Ch. 1–3, 9 / Part 1
A2	Vector derivatives and optimisation	15	15	Ch. 4–5 / Part 2
A3	Matrix derivatives and Gaussians	20	20	Ch. 6–7 / Part 3
A4	Backpropagation	17	20	Ch. 8 / Part 4
A5	Automatic differentiation	24	20	Ch. 8–9 / Part 5
A6	Attention, normalisation, flows	19	15	Ch. 10–12 / Part 6
	Total	**112**	**100**	

1.6 How to work

- **Paper first.** Derive each gradient with the differential method (Textbook Chapter 6): write $\mathrm{d}f$, push it into the form $\mathrm{tr}(G^{\mathsf{T}}\mathrm{d}X)$, read off G. Then check the *shape*: if the variable is $D \times K$, so is G.
- **Check it yourself before the tests do.** Once A1 is done, `numerical_gradient` and `relative_error` are your debugger:

```python
from mc_assignments.a1_matrix_functions import numerical_gradient, relative_error
value, G = trace_inv(X, A)
print(relative_error(G, numerical_gradient(lambda Z: trace_inv(Z, A)[0], X)))
```

  Around 10^{-9} or smaller means correct; 10^{-1} means a missing transpose or factor.
- **Test non-symmetric inputs.** Many textbook formulas assume a symmetric matrix. The tests perturb one entry at a time, which makes the input non-symmetric, so formulas like $\partial/\partial X \ln|X| = X^{-1}$ (true only for symmetric X) fail. Use the general forms ($X^{-\mathsf{T}}$).
- **One function at a time.** Run `pytest -k <function_name>` until it passes, then move on.
- **Try before you look.** The Lab Manual implements many of these ideas. Attempt each function from the textbook first; consult the manual only when stuck, then close it and write your own version. Several functions here are not in the manual at all: `block_logabsdet`, `ridge_solution`, `normalize_jacobian`, `gd_convergence_factor`, `gaussian_conditional`, `kl_gaussians`, the whole derivative catalogue of A3, the `Sigmoid` and `MSELoss` layers, `gradient_forward`, `batchnorm_forward`/`backward` and `top_eigenvector`.
- A separate archive, `matcalc_assignments_solutions.zip`, contains reference solutions. Keep it closed until you have a passing (or abandoned) attempt.

CHAPTER 2

A1 — Gradient Checking, Matrix Functions, Identities

File: `mc_assignments/a1_matrix_functions.py` **Points:** 10 **Reading:** Textbook Ch. 1–3 and 9.1, Lab Part 1.

Goals

Build the tool that checks every later assignment, then the matrix functions and identities that appear throughout Bishop: log-determinants, square roots, exponentials, low-rank updates and partitioned matrices.

Function	What it must do
`numerical_gradient(f, X, h=1e-6)`	Central differences $(f(X+hE)-f(X-hE))/2h$, entry by entry, for X of *any* shape. Returns an array shaped like X. Must not modify X.
`numerical_jacobian(f, x, h=1e-6)`	Central-difference Jacobian, inputs and outputs flattened row-major: shape (m, n) with $J_{ij} = \partial f_i / \partial x_j$.
`relative_error(a, b)`	$\|a-b\| \,/\, \max(\|a\|, \|b\|, 10^{-12})$ as a `float`.
`logdet_spd(S)`	$\ln\det S$ from the Cholesky factor; correct when $\det S$ under- or overflows; `ValueError` if S is not positive definite.
`sym_matrix_function(S, fn)`	$Q\,\mathrm{fn}(\Lambda)\,Q^{\mathsf{T}}$ for symmetric $S = Q\Lambda Q^{\mathsf{T}}$; `fn` acts on the 1-D array of eigenvalues.
`sqrtm_psd(S)`	The symmetric PSD square root; tiny negative eigenvalues from rounding count as 0.
`inv_sqrtm_pd(S)`	$S^{-1/2}$, the whitening matrix.
`expm(A, terms=20)`	$\exp A$ for *any* square A (not only symmetric), by the Taylor series with scaling and squaring; relative error $< 10^{-10}$ even for $\|A\| \approx 20$.
`sherman_morrison(Ainv, u, v)`	$(A + \boldsymbol{u}\boldsymbol{v}^{\mathsf{T}})^{-1}$ from A^{-1} in $O(n^2)$ operations.
`woodbury(Ainv, U, C, V)`	$(A+UCV)^{-1}$ from A^{-1}, solving only a $k \times k$ system.
`block_inverse(A, B, C, D)`	The full inverse of $\left[\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right]$ via the Schur complement of D.
`block_logabsdet(A, B, C, D)`	$\ln\lvert\det D\rvert + \ln\lvert\det(A - BD^{-1}C)\rvert$.

Hints.

- `numerical_gradient`: start with `X = np.array(X, dtype=float)` (a float copy), loop over `np.ndindex(X.shape)`, and restore each entry after perturbing it. An integer array would silently round $X + h$.
- `logdet_spd`: `np.linalg.cholesky` raises `np.linalg.LinAlgError` for a matrix that is not positive definite — catch it and raise `ValueError`. Never compute `np.log(np.linalg.det(S))`: $\det(0.1 I_{400}) = 10^{-400}$ underflows to 0.
- `expm`: with $\|A\| = 20$ the Taylor terms grow to about $20^{20}/20! \approx 4 \times 10^7$ before they shrink, and 20 terms are nowhere near enough. Pick s with $\|A\|/2^s \le \frac{1}{2}$ (any matrix norm, e.g. the largest absolute column sum), sum the series for $B = A/2^s$, then square the result s times: $\exp A = (\exp B)^{2^s}$.
- `sherman_morrison`: compute $A^{-1}\boldsymbol{u}$ and $\boldsymbol{v}^{\mathsf{T}}A^{-1}$ once each; the update is an outer product divided by $1 + \boldsymbol{v}^{\mathsf{T}}A^{-1}\boldsymbol{u}$ (Textbook Worked Example 3.3).
- `block_inverse`: `np.block([[P11, P12], [P21, P22]])` assembles the result (Worked Example 3.2).

What the tests check.

The textbook values: the gradient and Jacobian of Worked Examples 1.1–1.2, $\ln\det S$ of Worked Example 2.1, $\sqrt{S}$ of Worked Example 2.2, both exponentials of Worked Example 2.3, Sherman–Morrison of Worked Example 3.3 and the partitioned inverse and determinant of Worked Example 3.2. Gradients for X of shapes $(5,)$, $(3,4)$, $(2,3,2)$; that X is unchanged; Jacobians of linear maps; $\ln\det$ of $0.1 I_{400}$ and $10^3 I_{200}$; a `ValueError` for an indefinite matrix; $R^2 = S$ and $S^{-1/2} S S^{-1/2} = I$; $\det\exp M = e^{\operatorname{tr} M}$; a rotation by $\theta = 20$ and an SPD exponential to 10^{-9}; Woodbury with $n = 30$, $k = 3$ against `np.linalg.inv`.

Stretch goals (not graded).

- Plot the error of `numerical_gradient` on $f(x) = e^x$ against h from 10^{-1} to 10^{-12} and locate the V-shaped minimum (Textbook §9.1). Repeat with forward differences and with the complex step.
- Time `woodbury` against `np.linalg.inv` for $n = 1000$, $k = 5$ with diagonal A.

CHAPTER 3

A2 — Vector Derivatives and Optimisation

File: `mc_assignments/a2_vector_derivatives.py` **Points:** 15 **Reading:** Textbook Ch. 4–5, Lab Part 2.

Goals

Gradients, Jacobians and Hessians of the losses of Bishop's Chapters 4–5, computed stably; and the curvature analysis that explains when gradient descent converges and why Newton's method is fast.

Function	What it must do
`quadratic(x, A, b)`	$f = \boldsymbol{x}^\mathsf{T} A\boldsymbol{x} + \boldsymbol{b}^\mathsf{T}\boldsymbol{x}$ for square, *not necessarily symmetric* A. Returns `(f, gradient, Hessian)`.
`least_squares(w, Phi, t, lam=0.0)`	$E = \frac{1}{2}\lVert\Phi\boldsymbol{w} - \boldsymbol{t}\rVert^2 + \frac{\lambda}{2}\lVert\boldsymbol{w}\rVert^2$. Returns `(E, gradient, Hessian)`.
`ridge_solution(Phi, t, lam)`	$(\Phi^\mathsf{T}\Phi + \lambda I)^{-1}\Phi^\mathsf{T}\boldsymbol{t}$, computed with a solve.
`logsumexp(a, axis=-1)`	$\ln\sum_k e^{a_k}$ along `axis`, no overflow for $a_k \sim 10^4$.
`softmax(a, axis=-1)`	Softmax along `axis`, no overflow.
`softmax_jacobian(a)`	$\mathrm{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^\mathsf{T}$ at a 1-D vector $\boldsymbol{a}$.
`sigmoid(a)`	$1/(1+e^{-a})$ element-wise, never overflowing (inputs up to $\pm 10^4$).
`logistic_loss(w, Phi, t)`	$E = -\sum[t_n \ln y_n + (1-t_n)\ln(1-y_n)]$, $y_n = \sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}_n)$. Returns `(E, gradient, Hessian)`; finite for logits $\sim \pm 1000$.
`normalize_jacobian(x)`	Jacobian of $\boldsymbol{x}/\lVert\boldsymbol{x}\rVert$: $(I - \boldsymbol{x}\boldsymbol{x}^\mathsf{T}/\lVert\boldsymbol{x}\rVert^2)/\lVert\boldsymbol{x}\rVert$.
`gd_convergence_factor(H, lr)`	$\max_i \lvert 1 - \eta\lambda_i\rvert$ for gradient descent on $\frac{1}{2}\boldsymbol{w}^\mathsf{T}H\boldsymbol{w}$.
`optimal_learning_rate(H)`	$2/(\lambda_{\min} + \lambda_{\max})$.
`newton_logistic(Phi, t, n_steps=10)`	Newton/IRLS from $\boldsymbol{w} = \boldsymbol{0}$; returns $\boldsymbol{w}$ after `n_steps`.

Hints.

- For non-symmetric A, $\nabla(\boldsymbol{x}^\mathsf{T}A\boldsymbol{x}) = (A + A^\mathsf{T})\boldsymbol{x}$, not $2A\boldsymbol{x}$ (Textbook §4.1).
- Subtract the maximum before exponentiating (Worked Example 9.4). For `logsumexp`

with `keepdims=True` inside and a squeeze at the end, the same code works for any `axis`.

- `sigmoid`: use $e^{-|a|}$, which never overflows, and choose the formula by the sign of a with `np.where`.
- `logistic_loss`: the loss of one point is $\text{softplus}(a_n) - t_n a_n$ with $a_n = \boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}_n$ and $\text{softplus}(a) = \max(a, 0) + \log 1\text{p}(e^{-|a|})$. Never take $\ln y_n$ of a y_n that has rounded to 0 or 1. The Hessian is $\Phi^\mathsf{T} R \Phi$ with $R = \text{diag}(y_n(1 - y_n))$; build it as `Phi.T @ (Phi * r[:, None])` rather than forming R.
- The eigenvalues of a symmetric H: `np.linalg.eigvalsh(H)`, in increasing order.

What the tests check.

Worked Example 4.1 ($f = 17$, $\nabla f = (9, 13)$), the softmax Jacobian of Worked Example 4.2, the overflow example of Worked Example 9.4, the learning rates of Worked Example 5.1 ($\eta = 0.1$ gives factor 0.9, $\eta = 0.21$ gives 1.1) and the first Newton step $w_1 = -0.4$ of Worked Example 5.2. Gradients against finite differences and Hessians against finite-difference Jacobians of your gradient; a non-symmetric A; ridge regression with linearly dependent features (the gradient must vanish at your solution); the logsumexp gradient equals the softmax; the logistic Hessian is PSD; logits of ± 1000 (with floating-point warnings turned into errors); convergence of Newton to a zero gradient in 10–15 steps.

Stretch goals (not graded).

- Run gradient descent with `optimal_learning_rate` on quadratics with condition numbers 10, 100, 1000 and compare the observed rate with $(\kappa - 1)/(\kappa + 1)$.
- Make the data linearly separable and watch `newton_logistic`: what happens to $\|\boldsymbol{w}\|$, and why does adding $\frac{\lambda}{2}\|\boldsymbol{w}\|^2$ fix it?

CHAPTER 4

A3 — Matrix Derivatives and Gaussians

File: `mc_assignments/a3_matrix_derivatives.py` **Points:** 20 **Reading:** Textbook Ch. 6–7, Lab Part 3.

Goals

Derive, with the differential method, a catalogue of derivatives with respect to a whole matrix; then use them on the Gaussian: maximum likelihood, conditioning and the KL divergence used by variational autoencoders.

Every catalogue function returns `(value, gradient)`, the gradient shaped like X. Treat every entry of X as an independent variable — the tests perturb single entries, so your formulas must hold for non-symmetric X.

Function	What it must do
`trace_AX(X, A)`	$\operatorname{tr}(AX)$.
`trace_XtAX(X, A)`	$\operatorname{tr}(X^\top AX)$, A square and not necessarily symmetric; X may be rectangular.
`bilinear(X, a, b)`	$\boldsymbol{a}^\top X \boldsymbol{b}$ (X may be rectangular).
`logabsdet(X)`	$\ln \lvert\det X\rvert$ for any invertible X (either sign of $\det X$).
`det(X)`	$\det X$.
`trace_inv(X, A)`	$\operatorname{tr}(X^{-1}A)$.
`inv_bilinear(X, a, b)`	$\boldsymbol{a}^\top X^{-1}\boldsymbol{b}$.
`frobenius_lsq(X, A, B)`	$\lVert AX - B\rVert_F^2$.
`tanh_weighted(X, A)`	$\sum_{ij} A_{ij} \tanh X_{ij} = \operatorname{tr}(A^\top \tanh X)$.
`gaussian_nll(X, mu, Sigma)`	Negative log-likelihood of the rows of X ($N \times D$); returns `(value, d/dmu, d/dSigma)`.
`gaussian_mle(X)`	Maximum-likelihood $(\boldsymbol{\mu}, \Sigma)$, dividing by N.
`gaussian_conditional(mu, Sigma, idx_b, x_b)`	$(\boldsymbol{\mu}_{a\mid b}, \Sigma_{a\mid b})$ given the coordinates `idx_b` observed at `x_b`; a = the other coordinates in increasing order.
`kl_gaussians(mu1, S1, mu2, S2)`	$\mathrm{KL}(\mathcal{N}(\boldsymbol{\mu}_1, \Sigma_1) \,\|\, \mathcal{N}(\boldsymbol{\mu}_2, \Sigma_2))$ for full covariances.
`kl_diag_standard(mu, logvar)`	The VAE term $\mathrm{KL}(\mathcal{N}(\boldsymbol{\mu}, \operatorname{diag} e^{v}) \,\|\, \mathcal{N}(\mathbf{0}, I))$; returns `(value, d/dmu, d/dlogvar)`.

Hints.

- The identification rule: if $\mathrm{d}f = \operatorname{tr}(G^\top \mathrm{d}X)$ then $\partial f/\partial X = G$. So $\mathrm{d}\operatorname{tr}(AX) = \operatorname{tr}(A\mathrm{d}X)$ gives $G = A^\top$ — not A. Most bugs in this assignment are a missing transpose; a symmetric test matrix would hide them, which is why the tests use none.
- $\mathrm{d}X^{-1} = -X^{-1}\mathrm{d}X\,X^{-1}$ and $\mathrm{d}\ln|\det X| = \operatorname{tr}(X^{-1}\mathrm{d}X)$ (Theorem 6.3 and the formula sheet).
- Compute values with `np.linalg.slogdet` and `solve`, not with `cholesky`: Cholesky reads only one triangle of Σ, so when a test perturbs Σ_{12} alone, your value would not change and the finite-difference check would see a gradient of zero. (The Lab Manual tells the story of exactly this bug.)
- `gaussian_nll`: with Σ treated as a general matrix, the quadratic form $\boldsymbol{d}^\top\Sigma^{-1}\boldsymbol{d}$ has gradient $-\Sigma^{-\top}\boldsymbol{d}\boldsymbol{d}^\top\Sigma^{-\top}$ and $\ln|\Sigma|$ has gradient $\Sigma^{-\top}$. For symmetric Σ these reduce to the textbook's $-\frac{1}{2}\Sigma^{-1} + \frac{1}{2}\Sigma^{-1}S\Sigma^{-1}$ form.
- `gaussian_conditional`: `np.ix_(rows, cols)` extracts a sub-block; compute $\Sigma_{ab}\Sigma_{bb}^{-1}$ with a solve.

What the tests check.

Every catalogue gradient against finite differences on non-symmetric, rectangular inputs; values against direct NumPy formulas; Worked Example 6.1 ($\partial/\partial X \ln|X|$ for $X = \left[\begin{smallmatrix}2&1\\1&1\end{smallmatrix}\right]$) and Worked Example 6.2 ($\operatorname{tr}(X^\top AX)$ with non-symmetric A); a matrix with negative determinant. The Gaussian negative log-likelihood against the textbook formula and both gradients against finite differences; the three-point estimates of Worked Example 7.1 and a zero gradient at the MLE; the conditional of Worked Example 7.2 and a 5-dimensional conditional against the precision-matrix form; KL $= 0$ for equal Gaussians, a one-dimensional closed form, KL > 0 for random pairs; the VAE term of Worked Example 7.3 and its agreement with `kl_gaussians`.

Stretch goals (not graded).

- Fit a Gaussian by gradient descent on `gaussian_nll`, parameterising $\Sigma = LL^\top$ with lower-triangular L. Derive $\partial/\partial L$ by the chain rule and compare the result with `gaussian_mle`.
- Derive and test $\partial/\partial\boldsymbol{\mu}_1$ and $\partial/\partial\Sigma_1$ of `kl_gaussians`.

CHAPTER 5

A4 — Backpropagation

File: `mc_assignments/a4_backprop.py` **Points:** 20 **Reading:** Textbook Ch. 8, Lab Part 4.

Goals

Write the backward pass of each layer as a matrix formula, assemble the layers into a multilayer perceptron, and train it on the three-class spiral data set with nothing but your own gradients.

The layer interface. A mini-batch is a 2-D array with one example per *row*. `layer.forward(X)` returns the output and stores what the backward pass needs. `layer.backward(dY)` receives $\partial L/\partial Y$ (the shape of the output), stores parameter gradients in `layer.dW`, `layer.db`, and returns $\partial L/\partial X$. `params()` and `grads()` return lists of arrays in the same order; `params()` must return the parameter arrays themselves, so that an in-place update changes the layer.

Function	What it must do
`Linear(d_in, d_out, rng)`	$Y = XW + \mathbf{1}\boldsymbol{b}^{\mathsf{T}}$, W of shape $(d_{\text{in}}, d_{\text{out}})$ initialised as `rng.standard_normal((d_in, d_out)) * sqrt(1/d_in)`, $\boldsymbol{b} = \mathbf{0}$.
`ReLU`, `Tanh`, `Sigmoid`	Element-wise activations without parameters. $\text{ReLU}'(0) = 0$; Sigmoid stable for $\lvert z\rvert \le 10^4$.
`MSELoss`	`forward(Y, T)` returns $\frac{1}{2N}\lVert Y - T\rVert_F^2$; `backward()` returns $\partial L/\partial Y$.
`SoftmaxCrossEntropy`	`forward(A, y)`: mean cross-entropy of the row softmax of logits A with integer labels y, stable for logits $\sim 10^4$; `backward()` returns $\partial L/\partial A$.
`MLP(sizes, activation, seed)`	Linear–act–…–Linear with a softmax cross-entropy loss; one `rng` creates the Linear layers in order. Methods `forward`, `loss`, `backward`, `params`, `grads`.
`train(model, X, y, lr, epochs, momentum)`	Full-batch gradient descent with momentum, $\boldsymbol{v} \leftarrow \mu\boldsymbol{v} - \eta\boldsymbol{g}$, $\boldsymbol{p} \leftarrow \boldsymbol{p} + \boldsymbol{v}$, in place; returns the loss before each epoch's update.
`accuracy(model, X, y)`	Fraction of rows whose largest logit is the true label.

Hints.

- The linear layer (Theorem 8.1): $\bar{W} = X^{\mathsf{T}}\bar{Y}$, $\bar{\boldsymbol{b}} = \bar{Y}^{\mathsf{T}}\mathbf{1}$ (a column sum), $\bar{X} = \bar{Y}W^{\mathsf{T}}$. Check the shapes: each gradient has the shape of its variable, and that alone almost

determines the formula.

- Activations: $\bar{Z} = \bar{H} \odot h'(Z)$. Store what you need in `forward`: the mask $Z > 0$ for ReLU, the output H for tanh ($h' = 1 - H^2$) and sigmoid ($h' = H(1 - H)$).
- Softmax cross-entropy: $\partial L / \partial A = (S - T)/N$ with T the one-hot labels (Worked Example 8.2). Compute log-probabilities as $A - \mathrm{lse}(A)$ row by row; build $S - T$ with `G[np.arange(N), y] -= 1`.
- `train`: update with `p += v` (or `p -= …`), never `p = p + v`, which rebinds a local name and leaves the model unchanged. The test checks that the arrays are the same objects after training.

What the tests check.

The initialisation and forward pass of `Linear`; $\bar{X}$, $\bar{W}$, $\bar{\boldsymbol{b}}$ against finite differences; the two-layer ReLU network of Worked Example 8.1 entry by entry; each activation against finite differences, $\mathrm{ReLU}'(0) = 0$, the sigmoid at $\pm 10^4$; MSE and cross-entropy values and gradients, Worked Example 8.2, logits of 10^4; the parameter shapes and initialisation of an `MLP`; a complete gradient check of every parameter *and* the input of a 3-layer network with tanh and with ReLU; one and two steps of momentum exactly; and finally training: a $[2, 64, 64, 3]$ tanh network on 300 spiral points for 1500 epochs must cut the loss by a factor of more than 3 and reach at least 95% training accuracy (the reference solution reaches 99.3%, in about two seconds).

Stretch goals (not graded).

- Plot the decision regions of your trained network, as in the spiral figure of the Lab Manual.
- Add L_2 weight decay to `train` and a `Dropout` layer; check the backward pass of dropout with a fixed mask.
- Replace the momentum update by Adam and compare the loss curves.

CHAPTER 6

A5 — Automatic Differentiation

File: `mc_assignments/a5_autodiff.py` **Points:** 20 **Reading:** Textbook Ch. 8–9, Lab Part 5.

Goals

Build the two modes of automatic differentiation from scratch: forward mode with dual numbers, and reverse mode with a `Tensor` class that records a computational graph and backpropagates through it — the core of every deep-learning framework, in about 150 lines.

Part A — forward mode. A `Dual` holds `val` and `der` and represents $val + der\,\epsilon$ with $\epsilon^2 = 0$.

Function	What it must do
`Dual` arithmetic	`+ - * /` with Duals *and* numbers on either side (`2 * d`, `1 - d`, `1 / d`), unary minus, `d ** p` for a real number p. `__init__` is given.
`d_sin`, `d_cos`, `d_exp`, `d_log`	Elementary functions on Duals (and on plain floats).
`derivative(f, x)`	$f'(x)$ from one evaluation of f at $x + 1\epsilon$.
`gradient_forward(f, x)`	∇f for $f : \mathbb{R}^n \to \mathbb{R}$ by n forward passes; `f` takes a *list* of n Duals.

Part B — reverse mode. A `Tensor` holds `data` and `grad` (same shape, starting at 0). The constructor is given: `out._parents` is the tuple of input Tensors and `out._back` is a function of no arguments that *adds* `out.grad`'s contribution to each parent's `.grad`. `__add__` is implemented for you as the pattern to follow.

Function	What it must do
`unbroadcast(grad, shape)`	Sum `grad` over the axes NumPy broadcasting added (leading axes) or stretched (axes of length 1 in `shape`).
`Tensor` operations	`a + b`, `a - b`, `a * b`, `a / b`, `-a` (Tensors or numbers, with broadcasting); `a ** p`; `a @ b`; `a.T`; `a.sum(axis=None)`, `a.mean(axis=None)`; `exp`, `log`, `tanh`, `relu`, `sigmoid`.
`Tensor.backward()`	For a size-1 Tensor: add ones to its `.grad`, then call every node's `_back` once, in reverse topological order. Gradients *accumulate* over calls.
`cross_entropy(logits, y)`	Mean softmax cross-entropy as one primitive node: stable forward, backward $(S - T)/N$.

Hints.

- Dual multiplication: $(a + a'\epsilon)(b + b'\epsilon) = ab + (ab' + a'b)\epsilon$. Implement `__rsub__` as `other - self`, not `self - other`; the given `_lift` turns a number into a Dual with derivative 0.
- Order matters in reverse mode. A node that is used twice (as x in $x \cdot x$, or y in $y + y\,x$) must receive *both* contributions before it passes its own gradient on. A depth-first search that appends each node after its parents gives a topological order; walk it backwards. Use a `set` of visited `id`s, so that shared nodes are visited once.
- Broadcasting: if `X` is 5×3 and `b` has shape `(3,)`, then `(X + b).grad` is 5×3 but `b.grad` must be summed over the rows — that is what `unbroadcast` does. Multiplication needs it too.
- `sum(axis)` spreads the incoming gradient back over the summed axis: `np.expand_dims` and `np.broadcast_to` (or adding to an array of zeros) do it. `mean` is `sum` times $1/n$, where n is the size of the reduced axis.
- Write `a - b` as `a + (-b)` and `a / b` as `a * b ** -1`: fewer backward rules to get right.

What the tests check.

Worked Example 9.2 ($f(2) = 12$, $f'(2) = 14$) and Worked Example 9.3 ($\bar{x}_1 = 18$, $\bar{x}_2 = 6$); thirteen Dual expressions with numbers on both sides; a composite of sin, exp, log, cos to 10^{-12}; `gradient_forward`; `unbroadcast` for leading, stretched and scalar shapes; twelve Tensor expressions, each checked twice — its *value* against a plain-NumPy version of the same function and its *gradients* against finite differences of that NumPy version (so a wrong forward pass cannot hide behind a consistent backward pass); shared nodes and a diamond-shaped graph; accumulation over two `backward()` calls; a two-layer network; `cross_entropy` alone, at logits of 10^4, and inside a larger graph.

Stretch goals (not graded).

- Train the spiral MLP of A4 with your Tensor class and confirm that its gradients equal those of your A4 backward pass to the last bit (the Lab Manual explains why they are identical, not just close).
- Make `backward()` iterative instead of recursive, so that a chain of 10 000 operations does not exceed Python's recursion limit.
- Add `Tensor.__getitem__` (indexing) and a `max(axis)` reduction.

CHAPTER 7

A6 — Attention, Normalisation, Flows

File: `mc_assignments/a6_layers.py` **Points:** 15 **Reading:** Textbook Ch. 10–12, Lab Part 6.

Goals

The backward passes of the layers that define modern architectures — one self-attention head with optional causal masking, layer and batch normalisation — and the determinant and trace computations behind normalising flows, plus PCA as a constrained optimisation solved by gradient ascent.

Function	What it must do
`softmax_rows(S)`	Row-wise softmax; a row that is $-\infty$ except for one entry gives a one-hot row.
`softmax_rows_backward(A, dA)`	$\partial L/\partial S$ from $A = \mathrm{softmax}(S)$ and $\partial L/\partial A$.
`attention_forward(X, Wq, Wk, Wv, causal=False)`	$Y = \mathrm{softmax}(QK^\top/\sqrt{d_k})V$ with $Q = XW_q$, $K = XW_k$, $V = XW_v$; `causal`: token i attends only to $j \le i$. Returns `(Y, cache)`.
`attention_backward(dY, cache)`	Returns `(dX, dWq, dWk, dWv)`.
`layernorm_forward(X, gamma, beta, eps)`	Normalise each *row* (population variance), then $\gamma \odot \hat{x} + \beta$. Returns `(Y, cache)`.
`layernorm_backward(dY, cache)`	Returns `(dX, dgamma, dbeta)`.
`batchnorm_forward/backward`	The same, normalising each *column* over the batch.
`coupling_forward(x, Ws, bs, Wt, bt)`	Affine coupling: $z_A = x_A$, $z_B = x_B \odot e^{s} + t$ with $s = \tanh(x_A W_s + b_s)$, $t = x_A W_t + b_t$. Returns `(z, logdet)`, one log-determinant per row.
`coupling_inverse(z, …)`	The exact inverse.
`linear_flow_nll(X, A, b)`	Mean NLL of the rows of X under $x = Az + b$, $z \sim \mathcal{N}(0, I)$. Returns `(value, dA, db)`.
`hutchinson_trace(matvec, n, n_samples, rng)`	Mean of $\epsilon^\top M \epsilon$ over Rademacher vectors drawn as `rng.choice([-1.0, 1.0], size=n)`.
`rayleigh_quotient(S, x)`	$R = x^\top S x / x^\top x$ and its gradient.
`top_eigenvector(S, n_steps, lr, seed)`	Gradient ascent on R with renormalisation from a random unit vector; returns `(eigenvalue, unit vector)`.

Hints.

- Attention backward (Theorem 12.2), in order: $\bar{V} = A^\top \bar{Y}$; $\bar{A} = \bar{Y} V^\top$; $\bar{S}$ from the row-softmax rule $\bar{\boldsymbol{a}} = \boldsymbol{s} \odot (\bar{\boldsymbol{s}} - (\boldsymbol{s}^\top \bar{\boldsymbol{s}})\mathbf{1})$; $\bar{Q} = \bar{S}K/\sqrt{d_k}$, $\bar{K} = \bar{S}^\top Q/\sqrt{d_k}$; and X is used three times, so $\bar{X} = \bar{Q}W_q^\top + \bar{K}W_k^\top + \bar{V}W_v^\top$. Worked Example 12.1 is a complete hand-checkable case.
- Causal masking: set the masked scores to `-np.inf` (`np.tril` builds the mask). Then $A_{ij} = 0$ there, and the softmax backward rule automatically gives those entries zero gradient.
- Layer norm (Theorem 12.3) with $\boldsymbol{g} = \bar{\boldsymbol{y}} \odot \gamma$: $\bar{\boldsymbol{x}} = \big(\boldsymbol{g} - \mathrm{mean}(\boldsymbol{g}) - \hat{\boldsymbol{x}}\,\mathrm{mean}(\boldsymbol{g} \odot \hat{\boldsymbol{x}})\big)/\tilde{\sigma}$, means over the normalised axis. Batch norm is the same code with `axis=0`.
- Coupling layer: the Jacobian is triangular, so $\ln|\det J| = \sum_j s_j$ — no determinant needed (Worked Example 11.2).
- Linear flow: $\boldsymbol{z}_n = A^{-1}(\boldsymbol{x}_n - \boldsymbol{b})$ and $\partial/\partial A\left[\tfrac{1}{2}\|\boldsymbol{z}\|^2 + \ln|\det A|\right] = A^{-\top}(I - \boldsymbol{z}\boldsymbol{z}^\top)$, averaged over the data (Textbook §11.2).
- `top_eigenvector`: $\nabla R = 2(S\boldsymbol{x} - R\boldsymbol{x})/\boldsymbol{x}^\top\boldsymbol{x}$. Divide the step by a scale such as $\max|S_{ij}|$ so that one learning rate works for any S, and renormalise after each step.

What the tests check.

Stable row softmax with $-\infty$ entries; the softmax backward rule against finite differences; Worked Example 12.1 ($\bar{X} = (\frac{3}{4}, \frac{1}{2})$); the $1/\sqrt{d_k}$ scaling; all four attention gradients against finite differences with and without the causal mask; that token 0 attends only to itself and that changing later tokens leaves earlier outputs unchanged; normalised rows (layer norm) and columns (batch norm); the layer-norm Jacobian of Worked Example 12.2; gradients of X, γ, β for both norms. The coupling layer: forward values, exact inversion, $\ln|\det J|$ against a finite-difference Jacobian, triangular structure. The linear flow at the point of Worked Example 11.1 ($\ln p = -5.031$) and its gradients. Hutchinson: the specified random draws, exactness for a diagonal matrix, and an unbiased estimate. The Rayleigh gradient, zero at an eigenvector; the top eigenvector of Worked Example 10.2 ($\lambda = 3$) and of a random 6×6 matrix.

Stretch goals (not graded).

- Add multiple heads and an output projection to the attention layer, and check the backward pass.
- Stack four coupling layers with permutations in between and train them on two-moons data by maximising the log-likelihood (you will need the gradients of `coupling_forward` — derive them, or use your A5 Tensor class).
- Compare the variance of Hutchinson's estimator with Rademacher and with Gaussian vectors for a matrix with a large diagonal.

APPENDIX A

Troubleshooting

`ModuleNotFoundError: No module named 'mc_assignments'` Run `pytest` or `python grade.py` from the top-level folder (the one containing `pytest.ini`).

`gradient has shape (3, 4), expected (4, 3)` You returned the transpose. Check the layout convention: the gradient has the shape of the variable (Textbook Chapter 1).

`gradient disagrees with finite differences: relative error 7.1e-01` A wrong formula. Typical causes: A instead of A^T; a formula valid only for symmetric matrices; a missing factor $\frac{1}{2}$ or N; a missing term because a variable is used twice (attention's X, a reused Tensor).

A relative error of about 10^{-5} to 10^{-3} Usually correct mathematics evaluated at a kink (ReLU at 0) or a value computed from only one triangle of a matrix (Cholesky). See the hints of A3.

`FloatingPointError: overflow encountered in exp` Some tests turn floating-point warnings into errors. Use the stable forms: subtract the maximum, use $e^{-|a|}$, use `np.log1p`.

`test_rules` fails with a line number Your module imports a forbidden package (even if unused). Remove the import.

`TimeoutError: test took longer than 60 s` Usually an infinite loop, or a Tensor graph traversal that revisits shared nodes exponentially often. Keep a set of visited nodes.

The tests pass one by one but not all together You are probably modifying an input array in place, or storing state in a module-level variable. Copy inputs first.

APPENDIX B

The Test Files

File	Tests	Covers
`tests/test_a1_matrix_functions.py`	17	A1
`tests/test_a2_vector_derivatives.py`	15	A2
`tests/test_a3_matrix_derivatives.py`	20	A3
`tests/test_a4_backprop.py`	17	A4
`tests/test_a5_autodiff.py`	24	A5
`tests/test_a6_layers.py`	19	A6
`tests/conftest.py`	—	time limit, score report
`tests/_helpers.py`	—	finite differences, test data, rule checks

Reading the tests is allowed and encouraged: each test is a precise statement of a property your function must have, and many use the textbook's worked examples as data. The finite-difference helpers in `_helpers.py` are deliberately independent of your A1 code, so a bug in your gradient checker cannot hide a bug elsewhere.